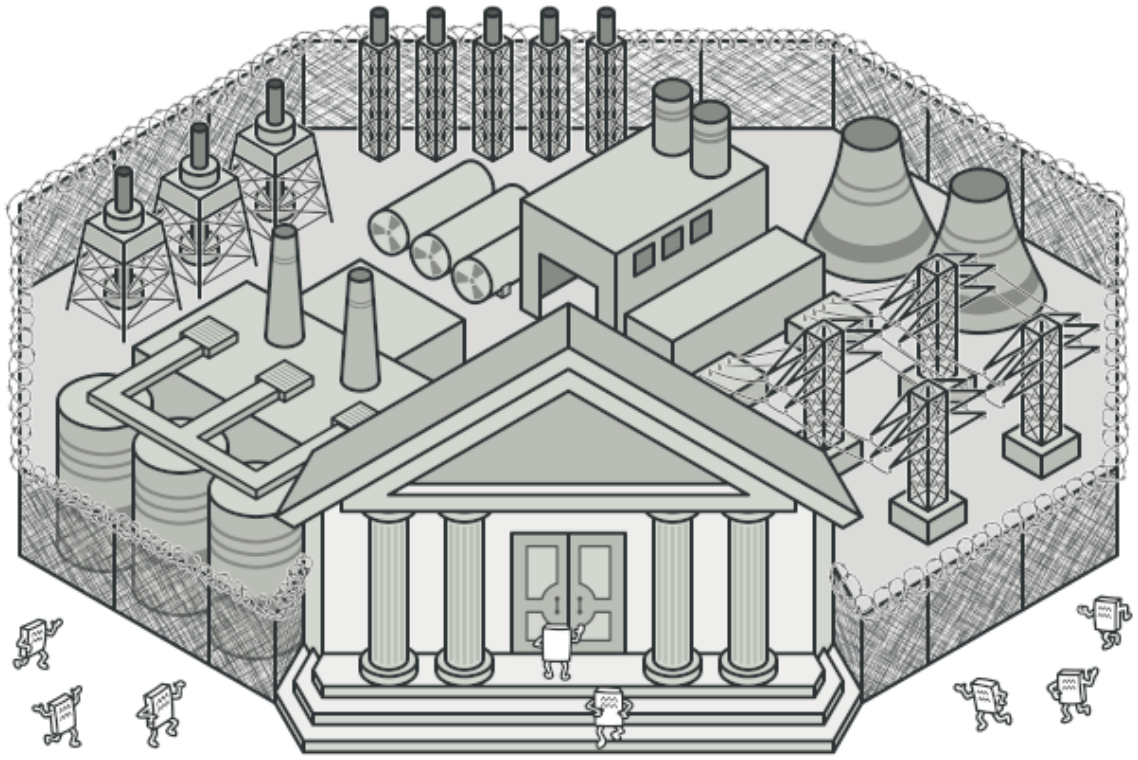
Facade

Absicht

Facade ist ein strukturelles Entwurfsmuster, das eine vereinfachte Schnittstelle zu einer Bibliothek, einem Framework oder einem anderen komplexen Satz von Klassen bereitstellt.



Problem

Stellen Sie sich vor, Ihr Code muss mit einer Vielzahl von Objekten einer komplexen Bibliothek oder eines Frameworks kompatibel sein. Normalerweise müssten Sie alle Objekte initialisieren, Abhängigkeiten im Auge behalten, Methoden in der richtigen Reihenfolge ausführen usw.

Dies hätte zur Folge, dass die Geschäftslogik Ihrer Klassen eng mit den Implementierungsdetails von Drittanbieterklassen verknüpft wäre, was das Verständnis und die Wartung erschwert.

Lösung

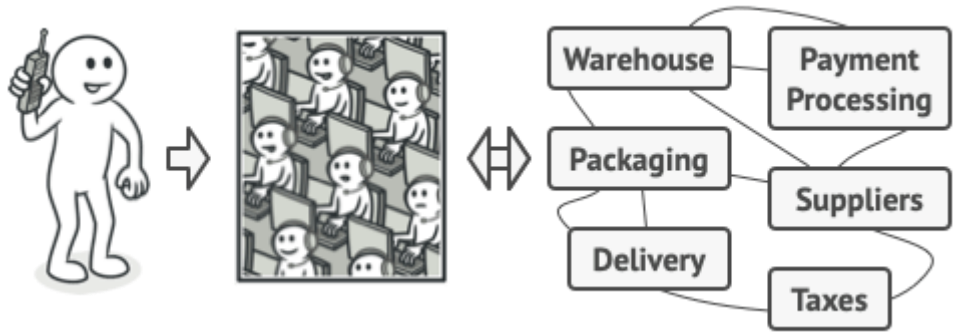
Eine Fassade ist eine Klasse, die eine einfache Schnittstelle zu einem komplexen Subsystem mit vielen beweglichen Teilen bereitstellt. Eine Fassade bietet möglicherweise im Vergleich zur direkten Arbeit mit dem Subsystem eingeschränkte Funktionalität. Sie enthält jedoch nur die Funktionen, die für Clients wirklich wichtig sind.

Eine Fassade ist praktisch, wenn Sie Ihre App in eine komplexe Bibliothek mit Dutzenden von Funktionen integrieren müssen, aber nur einen winzigen Teil ihrer Funktionalität benötigen.

Beispielsweise könnte eine App, die kurze, lustige Katzenvideos in soziale Medien hochlädt, eine professionelle Videokonvertierungsbibliothek gebrauchen. Alles, was sie wirklich benötigt, ist jedoch eine Klasse mit der Methode `encode(filename, format)`. Nachdem Sie

diese Klasse erstellt und mit der Videokonvertierungsbibliothek verknüpft haben, erhalten Sie Ihre erste Fassade.

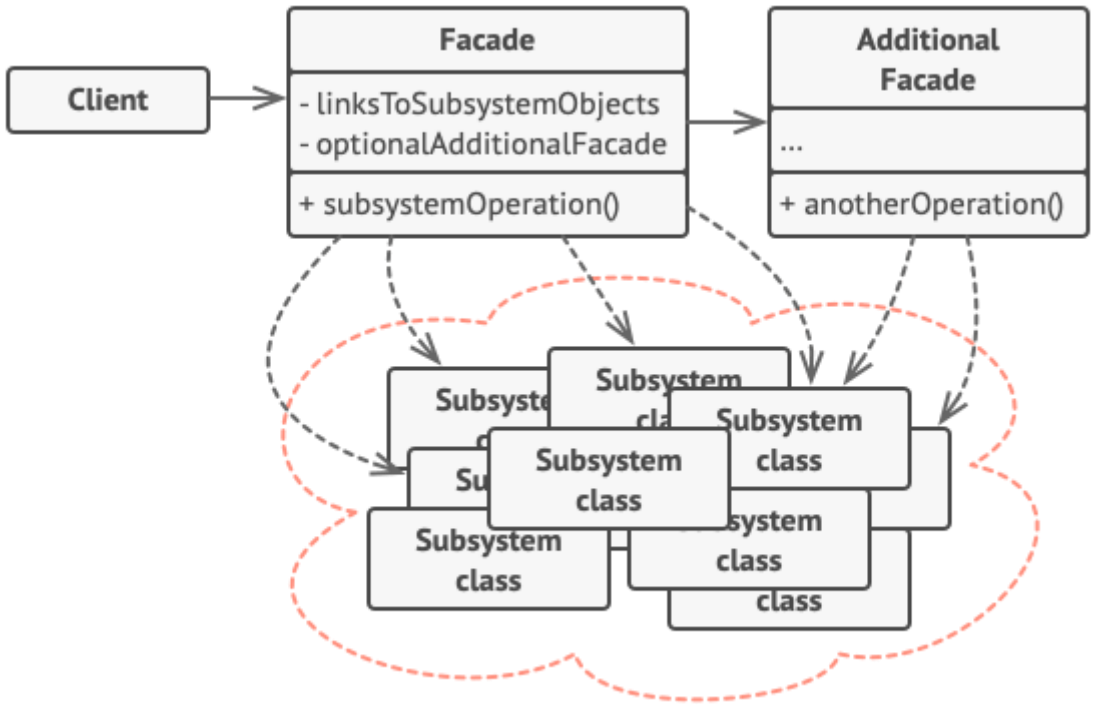
Analogie zur realen Welt



Telefonische Bestellung möglich.

Wenn Sie in einem Geschäft anrufen, um eine Bestellung aufzugeben, ist ein Mitarbeiter Ihr Ansprechpartner für alle Dienste und Abteilungen des Geschäfts. Er bietet Ihnen eine einfache Sprachschnittstelle zum Bestellsystem, zu Zahlungsgateways und verschiedenen Lieferdiensten.

Struktur



1. Die **Fassade** bietet bequemen Zugriff auf einen bestimmten Teil der Funktionalität des Subsystems. Sie weiß, wohin die Client-Anfrage zu leiten ist und wie alle beweglichen Teile zu bedienen sind.
2. Eine **zusätzliche** Fassadenklasse kann erstellt werden, um zu verhindern, dass eine einzelne Fassade durch irrelevante Elemente überladen wird, die sie zu einer weiteren komplexen Struktur machen könnten. Zusätzliche Fassaden können sowohl von Clients als auch von anderen Fassaden verwendet werden.

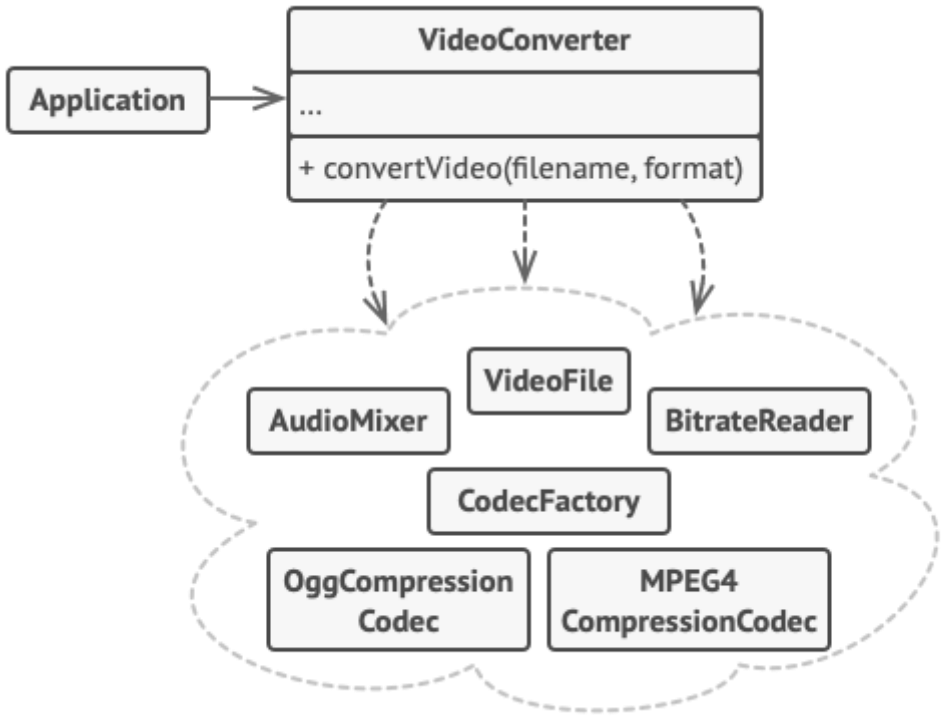
3. Das **komplexe Subsystem** besteht aus Dutzenden verschiedener Objekte. Damit sie alle sinnvolle Aufgaben erfüllen, müssen Sie sich eingehend mit den Implementierungsdetails des Subsystems befassen, z. B. mit der Initialisierung der Objekte in der richtigen Reihenfolge und der Bereitstellung von Daten im richtigen Format.

   Subsystemklassen sind sich der Existenz der Fassade nicht bewusst. Sie agieren innerhalb des Systems und arbeiten direkt miteinander.

4. Der **Client** verwendet die Fassade, anstatt die Subsystemobjekte direkt aufzurufen.

Pseudocode

In diesem Beispiel vereinfacht das **Facade** -Muster die Interaktion mit einem komplexen Videokonvertierungsframework.



Ein Beispiel für die Isolierung mehrerer Abhängigkeiten innerhalb einer einzelnen Fassadenklasse.

Anstatt Ihren Code direkt mit Dutzenden von Framework-Klassen arbeiten zu lassen, erstellen Sie eine Fassade-Klasse, die diese Funktionalität kapselt und vor dem Rest des Codes verbirgt. Diese Struktur hilft Ihnen außerdem, den Aufwand für Upgrades auf zukünftige Framework-Versionen oder den Austausch durch eine andere zu minimieren. Das Einzige, was Sie in Ihrer App ändern müssten, wäre die Implementierung der Fassade-Methoden.

```
// These are some of the classes of a complex 3rd-party video
// conversion framework. We don't control that code, therefore
// can't simplify it.

class VideoFile
// ...

class OggCompressionCodec
// ...

class MPEG4CompressionCodec
// ...

class CodecFactory
// ...

class BitrateReader
// ...

class AudioMixer
// ...


// We create a facade class to hide the framework's complexity
// behind a simple interface. It's a trade-off between
// functionality and simplicity.
class VideoConverter is
    method convert(filename, format):File is
        file = new VideoFile(filename)
        sourceCodec = (new CodecFactory).extract(file)
        if (format == "mp4")
            destinationCodec = new MPEG4CompressionCodec()
        else
            destinationCodec = new OggCompressionCodec()
        buffer = BitrateReader.read(filename, sourceCodec)
        result = BitrateReader.convert(buffer, destinationCodec)
        result = (new AudioMixer()).fix(result)
        return new File(result)

// Application classes don't depend on a billion classes
// provided by the complex framework. Also, if you decide to
// switch frameworks, you only need to rewrite the facade class.
class Application is
    method main() is
        convertor = new VideoConverter()
        mp4 = convertor.convert("funny-cats-video.ogg", "mp4")
        mp4.save()
```

Anwendbarkeit

Verwenden Sie das Facade-Muster, wenn Sie eine begrenzte, aber unkomplizierte Schnittstelle zu einem komplexen Subsystem benötigen.
Subsysteme werden mit der Zeit oft komplexer. Selbst die Anwendung von Designmustern führt typischerweise zur Erstellung weiterer Klassen. Ein Subsystem wird dadurch flexibler und lässt sich in verschiedenen Kontexten leichter wiederverwenden, erfordert aber gleichzeitig immer mehr Konfigurations- und Boilerplate-Code vom Client. The Facade versucht, dieses Problem zu lösen, indem es einen schnellen Zugriff auf die am häufigsten genutzten Funktionen des Subsystems ermöglicht, der den meisten Client-Anforderungen gerecht wird.

Verwenden Sie die Fassade, wenn Sie ein Subsystem in Schichten strukturieren möchten.
Erstellen Sie Fassaden, um Einstiegspunkte zu jeder Ebene eines Subsystems zu definieren. Sie können die Kopplung zwischen mehreren Subsystemen reduzieren, indem Sie die Kommunikation ausschließlich über Fassaden festlegen.

Kehren wir beispielsweise zu unserem Videokonvertierungs-Framework zurück. Es lässt sich in zwei Ebenen unterteilen: Video- und Audioebene. Für jede Ebene können Sie eine Fassade erstellen und die Klassen der einzelnen Ebenen über diese Fassaden miteinander kommunizieren lassen. Dieser Ansatz ähnelt stark dem **Mediator-** Muster.

So implementieren Sie

1. Prüfen Sie, ob eine einfachere Schnittstelle als die eines vorhandenen Subsystems möglich ist. Sie sind auf dem richtigen Weg, wenn diese Schnittstelle den Client-Code von vielen Klassen des Subsystems unabhängig macht.
2. Deklarieren und implementieren Sie diese Schnittstelle in einer neuen Fassadenklasse. Die Fassade leitet die Aufrufe des Clientcodes an die entsprechenden Objekte des Subsystems weiter. Die Fassade ist für die Initialisierung des Subsystems und die Verwaltung seines weiteren Lebenszyklus verantwortlich, sofern der Clientcode dies nicht bereits übernimmt.
3. Um den vollen Nutzen aus dem Muster zu ziehen, sollte der gesamte Clientcode ausschließlich über die Fassade mit dem Subsystem kommunizieren. Dadurch ist der Clientcode vor Änderungen im Subsystemcode geschützt. Wenn beispielsweise ein Subsystem auf eine neue Version aktualisiert wird, müssen Sie nur den Code in der Fassade ändern.
4. Wenn die Fassade **zu groß** wird , sollten Sie einen Teil ihres Verhaltens in eine neue, verfeinerte Fassadenklasse extrahieren.

Für und Wider

- Sie können Ihren Code von der Komplexität eines Subsystems isolieren.
- Eine Fassade kann zu **einem Gottobjekt** werden , das mit allen Klassen einer App gekoppelt ist.

Beziehungen zu anderen Mustern

- **Facade** definiert eine neue Schnittstelle für vorhandene Objekte, während **Adapter** versucht, die vorhandene Schnittstelle nutzbar zu machen. *Adapter* umschließt in der Regel nur ein Objekt, während *Facade* mit einem ganzen Subsystem von Objekten arbeitet.
- **Abstract Factory** kann als Alternative zu **Facade** dienen , wenn Sie nur die Art und Weise verbergen möchten, wie die Subsystemobjekte vor dem Clientcode erstellt werden.
- **Flyweight** zeigt, wie man viele kleine Objekte erstellt, während **Facade** zeigt, wie man ein einzelnes Objekt erstellt, das ein ganzes Subsystem darstellt.
- **Facade** und **Mediator** haben ähnliche Aufgaben: Sie versuchen, die Zusammenarbeit zwischen vielen eng gekoppelten Klassen zu organisieren.
    - *Facade* definiert eine vereinfachte Schnittstelle zu einem Objektsubsystem, führt jedoch keine neuen Funktionen ein. Das Subsystem selbst kennt die Facade nicht. Objekte innerhalb des Subsystems können direkt kommunizieren.
    - *Mediator* zentralisiert die Kommunikation zwischen den Systemkomponenten. Die Komponenten kennen nur das Mediator-Objekt und kommunizieren nicht direkt miteinander.
- Eine **Facade** -Klasse kann oft in ein **Singleton** umgewandelt werden , da in den meisten Fällen ein einzelnes Fassadenobjekt ausreicht.
- **Facade** ähnelt **Proxy** darin, dass beide eine komplexe Entität puffern und selbst initialisieren. Im Gegensatz zu Facade *verfügt Proxy* über dieselbe Schnittstelle wie sein Serviceobjekt, wodurch sie austauschbar sind.